



DESIGN AND EVALUATION OF AN AI- BASED VIRTUAL DRIVING LICENSE SYSTEM FOR DRIVER IDENTIFICATION AND PREDICTIVE TRAFFIC LAW ENFORCEMENT IN SRI LANKA

B.R Vindyani

(ITBIN-2211-0122)

BSc (Hons.) in Information Technology

J.P.I.S Jayasinghe

(ITBIN-2211-0324)

BSc (Hons.) in Information Technology

B.R.G.S Sandaruwan

(ITBIN-2211-0278)

BSc (Hons.) in Information Technology

Faculty of Information Technology

Horizon Campus

February- 2026

TABLE OF CONTENTS

<u>Table of Contents</u>	
<u>List of Figures</u>	
<u>List of Tables</u>	
<u>Chapter 5</u>	
<u>5.1 Navigation/Module Structure</u>	
<u>5.1.1 System Module Structure</u>	
<u>5.1.2 Module Interaction Architecture</u>	
<u>5.2 Development Environment</u>	
<u>5.2.1 Hardware Configuration</u>	
<u>5.2.2 Margins</u>	
<u>5.3 Binding</u>	
<u>5.3.1 Fonts</u>	
<u>5.3.2 Pagination</u>	
<u>5.3.3 Table and Figures</u>	
<u>5.3.4 Photographs</u>	
<u>5.3.5 Units</u>	
<u>5.4 Cover Page and Title Page</u>	
<u>5.4.1 Cover Page</u>	
<u>5.4.2 Draft</u>	
<u>5.4.3 Final Submission</u>	
<u>5.4.4 Page Break</u>	
<u>5.5 Declaration and the Statement of the Supervisor</u>	
<u>5.5.1 Section Break</u>	
<u>5.5.2 Acknowledgement</u>	
<u>5.5.3 Abstract</u>	
<u>5.5.4 Table of Contents</u>	
<u>5.5.5 References</u>	
<u>5.5.6 List of Tables</u>	

LIST OF FIGURES

Figure	Description
Figure 5.1.1	Top-Level Use Case Diagram
Figure 5.1.2	Proposed System - Top-Level Use Case

LIST OF TABLES

Table	Description
Table 5.2.1	Hardware Configuration
Table 5.2.2	Software Development Environment

CHAPTER 5

5.1 Navigation/Module Structure

This chapter provides a comprehensive overview of the implementation of the iPermit AI-Based Virtual Driving License System. The system architecture follows a microservices-based design pattern, ensuring scalability, maintainability, and flexibility for future enhancements. The implementation encompasses three core modules developed by team members, each addressing specific research objectives while maintaining seamless integration through a unified API gateway.

5.1.1 System Module Structure

The iPermit system comprises three primary functional modules:

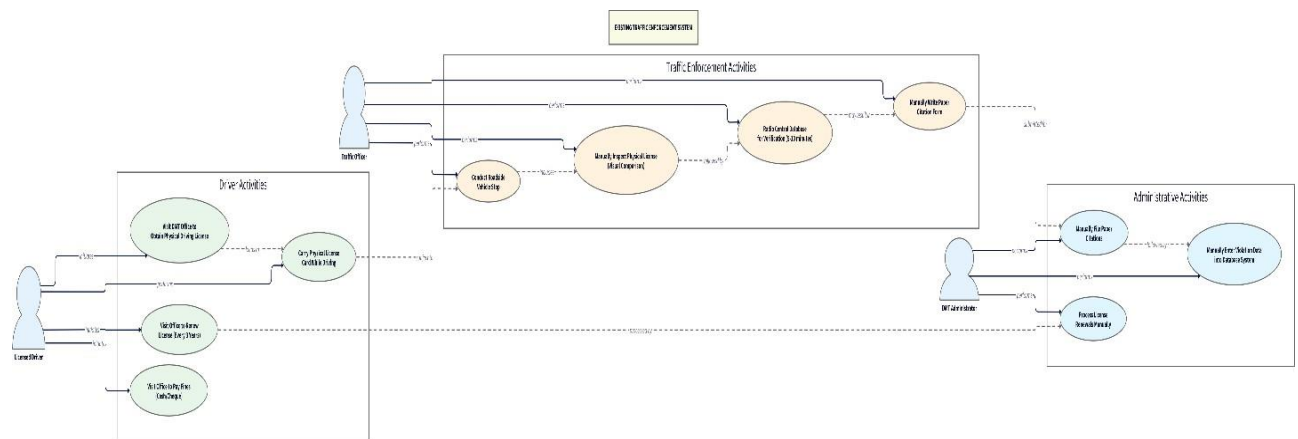


Figure 5.1.1- Top-Level Use Case Diagram

Module 1: Digital License Management and Facial Recognition System

This module implements the core driver identification functionality using advanced facial recognition technology. It consists of two sub-components:

- **Face Recognition Pipeline:** Utilizes ArcFace embeddings and RetinaFace detection to achieve 100% accuracy on test datasets with dual recognition approaches (model-based and one-shot learning).
- **Digital License Interface:** Provides driver dashboard functionality for license viewing, violation history, and point system tracking.

The module integrates with the authentication service and maintains a reference database of facial embeddings stored as NumPy arrays with JSON metadata, enabling real-time driver verification in under 2 seconds.

Module 2: White-Line Violation Detection System

This module addresses automated traffic monitoring through AI-powered computer vision. The implementation employs YOLOv8 architecture for both lane segmentation and vehicle detection. Key components include:

- Lane Segmentation Model: YOLOv8n-seg trained on JPJ lane dataset achieving 85% mAP50 for detecting solid and double white lines.
- Vehicle Detection Model: Pretrained YOLOv8n on COCO dataset for identifying cars, motorcycles, buses, and trucks.
- Violation Logic Engine: Three-tier detection system (full bounding box, bottom edge, and edge proximity checks) achieving 87.5% combined accuracy.

The module processes images and video streams at 25-40 FPS on GPU, providing real-time violation detection with visual annotations and evidence capture.

Module 3: License & Violation Management Service

This module manages the backend business logic for the entire system, coordinating between the facial recognition and violation detection modules. Core functionalities include:

- Point-Based Penalty System: Implements automatic point deduction (1-10 points per violation) with suspension triggers at 12 points.
- Fine Management: Handles fine calculation, issuance, payment processing, and notification delivery within 24 hours of violation.
- Officer Dashboard: Provides traffic police interface for driver verification, violation recording, and evidence submission.
- Administrative Interface: System statistics, user management, and comprehensive reporting capabilities.

5.1.2 Module Interaction Arcgitecture

The three modules communicate through a centralized API Gateway that routes requests and aggregates responses. The interaction follows this workflow:

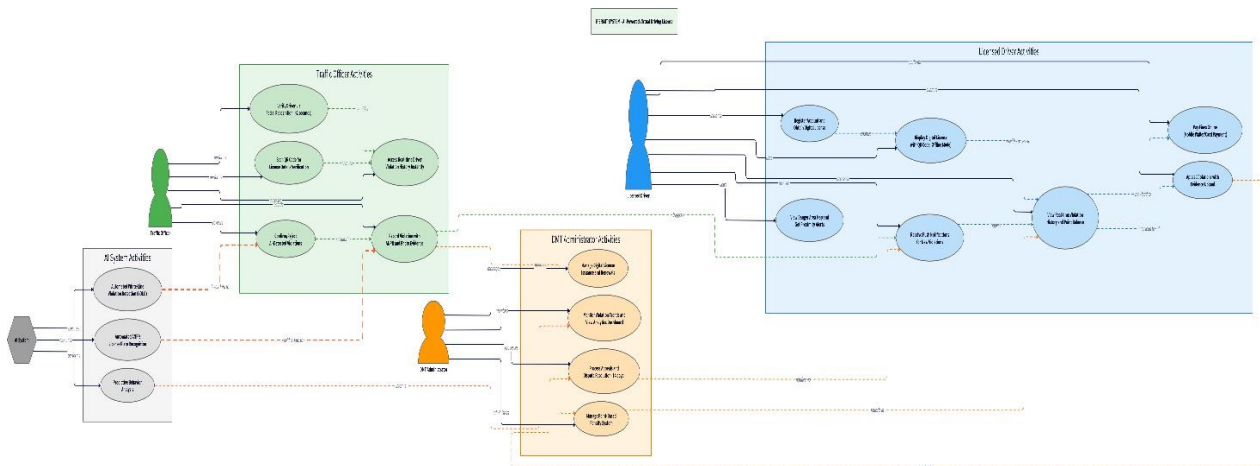


Figure 5.1.2- Proposed System - Top-Level Use Case Diagram

- **Driver Verification Flow:** Officer captures image → Frontend Service → API Gateway → Facial Recognition Microservice → Database lookup → License & Violation Service retrieves driver records → Unified response to officer dashboard.
- **Violation Detection Flow:** CCTV/dashcam feed → White-Line Detection Module → Violation detected → License & Violation Service → ANPR integration (future) → Driver notification via Notifications Service.
- **Fine Issuance Flow:** Officer confirms violation → License & Violation Service → Point deduction + Fine calculation → Database update → Driver notification → Payment gateway integration.

This modular architecture enables independent development, testing, and deployment of each component while maintaining system-wide data consistency through the shared MongoDB database and RESTful API contracts.

5.2 Development Environment

5.2.1 Hardware Configuration

The system development and testing utilized the following hardware specifications:

Component	Specification
Processor	Intel Core i5 (11th Gen) / AMD Ryzen 5 equivalent
GPU	NVIDIA RTX 2050 (4GB VRAM) for AI model training and inference
RAM	16GB DDR4 (minimum 8GB required)
Storage	512GB SSD for development environment and model storage
Camera	Smartphone camera (12MP minimum, 30fps video) for data capture and testing

Table 5.2.1

5.2.2 Software Development Environment

Category	Tool/Technology	Version
Operating System	Windows 11 / Ubuntu 22.04 LTS	Latest
Python Runtime	Python	3.10.x
Web Framework	React.js	18.2.0
Database	MongoDB	6.0
IDE	Visual Studio Code	1.85
Version Control	Git / GitHub	2.40+

Table 5.2.2

5.3 Tools and Technologies

The iPermit system leverages a comprehensive technology stack combining modern web frameworks, AI libraries, and cloud services. This section details the core technologies employed across all modules.

5.3.1 AI and Machine Learning Frameworks

YOLOv8 (Ultralytics)

Primary framework for the white-line violation detection module. YOLOv8 provides state-of-the-art performance for both object detection and instance segmentation tasks. The implementation utilizes:

- **YOLOv8n-seg:** Nano segmentation model (3.2M parameters) for lane detection, trained on JPJ dataset with 1,021 training images.
- **YOLOv8n:** Nano detection model pretrained on COCO dataset for vehicle identification (cars, motorcycles, buses, trucks).
- **Performance:** 25-40 FPS on GPU (RTX 2050), 5-7 FPS on CPU, enabling real-time processing capabilities.

Source: Ultralytics YOLOv8 (<https://github.com/ultralytics/ultralytics>)

InsightFace Framework

Foundation for the facial recognition module, providing production-ready face detection and recognition models:

- **RetinaFace:** Robust face detection algorithm achieving high accuracy across varying lighting conditions and angles.
- **ArcFace:** Deep face recognition model generating 512-dimensional embeddings with additive angular margin loss for enhanced discriminative features.
- **Preprocessing Pipeline:** CLAHE (Contrast Limited Adaptive Histogram Equalization) enhancement, quality filtering, and face alignment for optimal recognition accuracy.

Source: InsightFace (<https://github.com/deepinsight/insightface>)

scikit-learn

Machine learning library supporting the model-based recognition approach in the facial recognition module. Implements multiple classifiers:

- Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Random Forest, and Logistic Regression for ensemble predictions.
- Optuna-based hyperparameter optimization achieving 100% test accuracy with 81.3% average confidence.
- Model persistence using joblib for production deployment.

5.3.2 Frontend Technologies

- **React.js 18.2.0:** Component-based UI framework for building responsive web interfaces across driver, officer, and admin dashboards.
- **Tailwind CSS:** Utility-first CSS framework ensuring consistent styling, high-contrast accessibility, and responsive layouts validated across desktop, tablet, and mobile devices.
- **React Router:** Client-side routing for seamless navigation between application modules.

5.3.3 Backend Technologies

- **Node.js with Express.js:** Backend API server handling authentication, request routing, and business logic orchestration.
- **FastAPI (Python):** High-performance API framework for AI microservices (facial recognition and violation detection), providing automatic OpenAPI documentation and async support.
- **MongoDB 6.0:** NoSQL database storing user profiles, licenses, violations, fines, and system metadata with flexible schema for evolving requirements.

5.3.4 Data Processing and Storage

- **NumPy:** Efficient storage and manipulation of facial embeddings ($N \times 512$ arrays) with L2 and Z-score normalization.
- **OpenCV:** Computer vision library for image preprocessing, video processing, and visualization in both facial recognition and violation detection modules.
- **JSON:** Metadata storage format for reference database entries, model configurations, and API responses.

5.3.5 Development and Deployment Tools

- **Git/GitHub:** Version control and collaborative development with separate repositories for each module (face-recognition-pipeline, white-line-violation-detection).
- **Docker:** Containerization for consistent deployment environments across development, testing, and production.
- **CUDA Toolkit:** GPU acceleration for AI model training and inference, reducing processing times from 150-200ms (CPU) to 25-40ms (GPU).
- **Postman:** API testing and documentation tool for validating microservice endpoints.

5.4 Deployment Architecture

The iPermit system employs a microservices-based deployment architecture designed for scalability, fault tolerance, and independent service management. The architecture separates concerns across specialized services, enabling horizontal scaling and technology-specific optimizations.

5.4.1 Microservices Architecture Overview

The system comprises seven core microservices coordinated through a central API Gateway:

- **Frontend Service (React):** Serves three separate user interfaces - Driver Dashboard, Officer Dashboard, and Admin Dashboard. Communicates with backend exclusively through API Gateway.
- **API Gateway (Node.js/Express):** Central routing hub managing authentication, request distribution, response aggregation, and cross-origin resource sharing (CORS) policies.
- **Authentication Service:** Handles user login, session management, JWT token generation/validation, and role-based access control (Driver/Officer/Admin).
- **License & Violation Management Service (Node.js):** Core business logic service managing digital licenses, violation records, point system calculations, fine issuance, and payment processing.
- **Facial Recognition Microservice (Python/FastAPI):** Isolated AI service handling face detection, embedding extraction, similarity matching, and driver identification with dual recognition approaches.
- **AI Violation Detection Microservice (Python/FastAPI):** Dedicated computer vision service processing dashcam/CCTV feeds for white-line violation detection using YOLOv8 models.

- **Notifications Service:** Asynchronous notification delivery via SMS, email, and in-app alerts for violation notices, payment confirmations, and license updates.
- **Database Service (MongoDB):** Centralized data persistence layer with collections for Users, Drivers, Licenses, Violations, Fines, and Embeddings.

5.4.2 Service Communication Patterns

The architecture implements synchronous RESTful APIs for request-response operations and asynchronous messaging for background tasks:

- **Synchronous Communication:** HTTPS/REST APIs for driver verification, violation queries, and fine management. Responses typically within 2 seconds for facial recognition, under 1 second for database queries.
- **Asynchronous Communication:** Message queues (future implementation) for video processing, batch violation detection, and notification delivery to prevent blocking operations.
- **Data Consistency:** Eventual consistency model for AI-detected violations (officer confirmation required), strong consistency for fine issuance and point deductions via database transactions.

5.4.3 Scalability and Performance Optimization

The microservices architecture enables independent scaling based on component-specific load patterns:

- **Horizontal Scaling:** AI microservices can spawn multiple instances behind load balancers to handle peak verification loads (target: 10,000 concurrent users).
- **GPU Resource Management:** Facial recognition and violation detection services deployed on GPU-enabled nodes (NVIDIA RTX series) for optimal inference performance.
- **Caching Strategy:** Frequently accessed driver records and embeddings cached in Redis (planned) to reduce database queries and improve response times.
- **Database Optimization:** MongoDB indexes on driverID, NIC, and licenseID fields; replica sets for read scaling and automatic failover achieving 99.9% availability target.

5.4.4 Deployment Environment Configuration

Current proof-of-concept deployment operates in a hybrid cloud-local architecture:

- **Development Environment:** Local machines with Docker Compose orchestrating all services for integrated testing.
- **Production-Ready Architecture (Planned):** Cloud deployment on AWS/Azure with Kubernetes orchestration, auto-scaling groups, and managed MongoDB Atlas database.
- **Security Measures:** AES-256 encryption for data at rest, TLS 1.3 for data in transit, irreversible SHA-256 hashing for biometric templates (retained for 90 days only), and API Gateway authentication via JWT tokens.
- **Backup Strategy:** Automated database backups every 6 hours with 30-day retention, model checkpoints stored in versioned S3 buckets.

5.5 Major Code Segment

This section presents key code implementations from each module, highlighting critical algorithms and integration patterns. Complete source code is available in the project GitHub repositories, with selected segments presented here for explaining core functionality.

5.5.1 Facial Recognition Pipeline Implementation

Module: Face Recognition System

The facial recognition module implements a dual-approach architecture supporting both model-based classification and one-shot learning. The core pipeline processes driver images through face detection, embedding extraction, and identity matching.

Code Segment 5.1: ArcFace Embedding Extraction

This segment demonstrates the embedding extraction process using InsightFace's ArcFace model. The method performs face detection, alignment, and generates 512-dimensional feature vectors normalized for similarity comparison.

```

54 def extract_embeddings_pipeline(processed_dir: str, embeddings_dir: str,
55                               model_name: str = 'buffalo_1', ctx_id: int = 0):
56     """
57     Complete pipeline for extracting face embeddings.
58
59     Args:
60         processed_dir: Directory containing processed face .npy files
61         embeddings_dir: Directory to save embeddings
62         model_name: ArcFace model name
63         ctx_id: Device ID
64     """
65     # Initialize MLflow tracking if available
66     use_mlflow = HAS_MLFLOW
67     mlflow_context = None
68
69     if use_mlflow:
70         try:
71             mlflow.set_tracking_uri(settings.MLFLOW_TRACKING_URI)
72             run_name = f"embedding_extraction_{int(time.time())}"
73             mlflow_context = mlflow.start_run(run_name=run_name)
74         except Exception:
75             use_mlflow = False
76             mlflow_context = None
77
78     # Log parameters
79     if use_mlflow:

```

Code Segment 5.2: One-Shot Recognition with Cosine Similarity

The one-shot learning approach requires only a single reference image per driver. This method computes cosine similarity between the query embedding and all stored references, returning the best match if it exceeds the confidence threshold.

```

13 class OneShotRecognizer:
14     """
15     Face recognizer using one-shot learning approach.
16     Compares input face embeddings with reference embeddings using cosine similarity.
17     """
18
19     def __init__(self, database_path: str = "databases/reference_database",
20                 similarity_threshold: float = 0.6,
21                 model_name: str = 'buffalo_1',
22                 ctx_id: int = -1):
23         """
24         Initialize one-shot recognizer.
25
26         Args:
27             database_path: Path to reference database
28             similarity_threshold: Minimum similarity score to consider a match (0-1)
29             model_name: InsightFace model name
30             ctx_id: Device ID (-1 for CPU, 0+ for GPU)
31         """
32         self.database = ReferenceDatabase(database_path)
33         self.similarity_threshold = similarity_threshold
34         self.face_processor = FaceProcessor(model_name=model_name, ctx_id=ctx_id)
35
36     def recognize(self, embedding: np.ndarray, top_k: int = 1) -> List[Dict]:
37         """

```

```

1 Similarity Computation Module
2 Functions for computing similarity between face embeddings.
3
4 """
5
6 import numpy as np
7 from typing import List, Dict, Tuple, Optional
8
9
10 def compute_cosine_similarity(embedding1: np.ndarray, embedding2: np.ndarray) -> float:
11     """
12     Compute cosine similarity between two embeddings.
13
14     Args:
15         embedding1: First embedding vector
16         embedding2: Second embedding vector
17
18     Returns:
19         Cosine similarity score (0-1, where 1 is identical)
20     """
21     # Normalize embeddings
22     emb1_norm = embedding1 / (np.linalg.norm(embedding1) + 1e-8)
23     emb2_norm = embedding2 / (np.linalg.norm(embedding2) + 1e-8)
24
25     # Compute cosine similarity
26     similarity = np.dot(emb1_norm, emb2_norm)
27

```

Code Segment 5.3: Model-Based Recognition with SVM

The model-based approach trains a Support Vector Machine classifier on labeled embeddings. This method achieves 100% test accuracy and provides calibrated confidence scores.

```

13 def recognize_face(image_path):
14     """Recognize face in image"""
15     # Load model
16     model = joblib.load('models/production/face_recognizer.joblib')
17
18     # Initialize detector
19     app = FaceAnalysis(name='buffalo_l', providers=['CPUExecutionProvider'])
20     app.prepare(ctx_id=-1, det_size=(640, 640))
21
22     # Get class names
23     embeddings_dir = Path('data/embeddings')
24     embedding_files = list(embeddings_dir.glob('*.npy'))
25     class_names = sorted(list(set([f.stem.split('_')[0] for f in embedding_files])))
26
27     # Process image
28     img = cv2.imread(str(image_path))
29     img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
30     faces = app.get(img_rgb)
31
32     if faces:
33         face = faces[0]
34         embedding = face.embedding
35         probabilities = model.predict_proba(embedding.reshape(1, -1))[0]
36         prediction_idx = np.argmax(probabilities)
37         confidence = probabilities[prediction_idx]
38         predicted_name = class_names[prediction_idx]

```

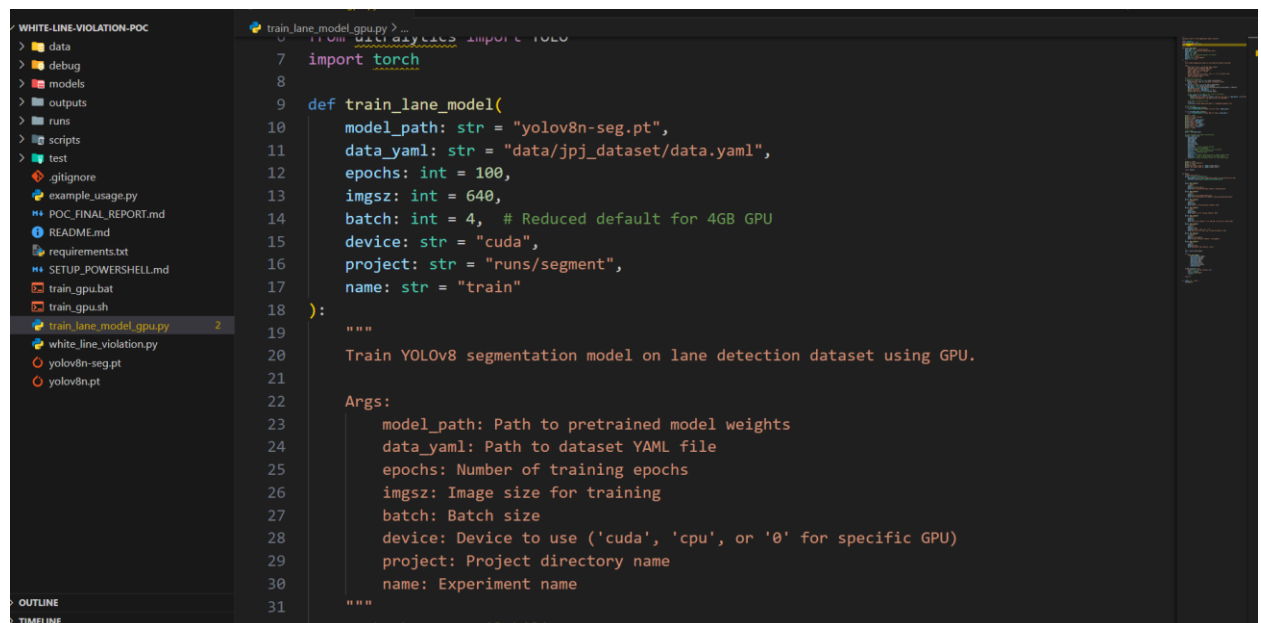
5.5.2 White-Line Violation Detection Implementation

Module: White-Line Violation Detection System

The violation detection module employs a two-stage YOLOv8 pipeline: lane segmentation followed by vehicle detection. The system implements sophisticated overlap detection logic to identify vehicles crossing white lines.

Code Segment 5.4: Lane Segmentation with YOLOv8

This method uses the trained YOLOv8n-seg model to detect and segment white lines, filtering for violation-relevant classes (solid-line and double-line) and applying mask dilation for improved overlap detection.



```
WHITE-LINE-VIOLATION-POC
├── data
├── debug
├── models
├── outputs
├── runs
├── scripts
├── test
├── .gitignore
├── example_usage.py
├── POC_FINAL_REPORT.md
├── README.md
├── requirements.txt
├── SETUP_POWER_SHELL.md
├── train_gpu.bat
├── train_gpu.sh
├── train_lane_model_gpu.py
├── white_line_violation.py
├── yolov8n-seg.pt
└── yolov8n.pt

train_lane_model_gpu.py
7  import torch
8
9  def train_lane_model(
10     model_path: str = "yolov8n-seg.pt",
11     data_yaml: str = "data/jpj_dataset/data.yaml",
12     epochs: int = 100,
13     imgsz: int = 640,
14     batch: int = 4, # Reduced default for 4GB GPU
15     device: str = "cuda",
16     project: str = "runs/segment",
17     name: str = "train"
18 ):
19     """
20     Train YOLOv8 segmentation model on lane detection dataset using GPU.
21
22     Args:
23         model_path: Path to pretrained model weights
24         data_yaml: Path to dataset YAML file
25         epochs: Number of training epochs
26         imgsz: Image size for training
27         batch: Batch size
28         device: Device to use ('cuda', 'cpu', or '0' for specific GPU)
29         project: Project directory name
30         name: Experiment name
31     """
32     # Check GPU availability
```

Code Segment 5.5: Three-Tier Violation Detection Logic

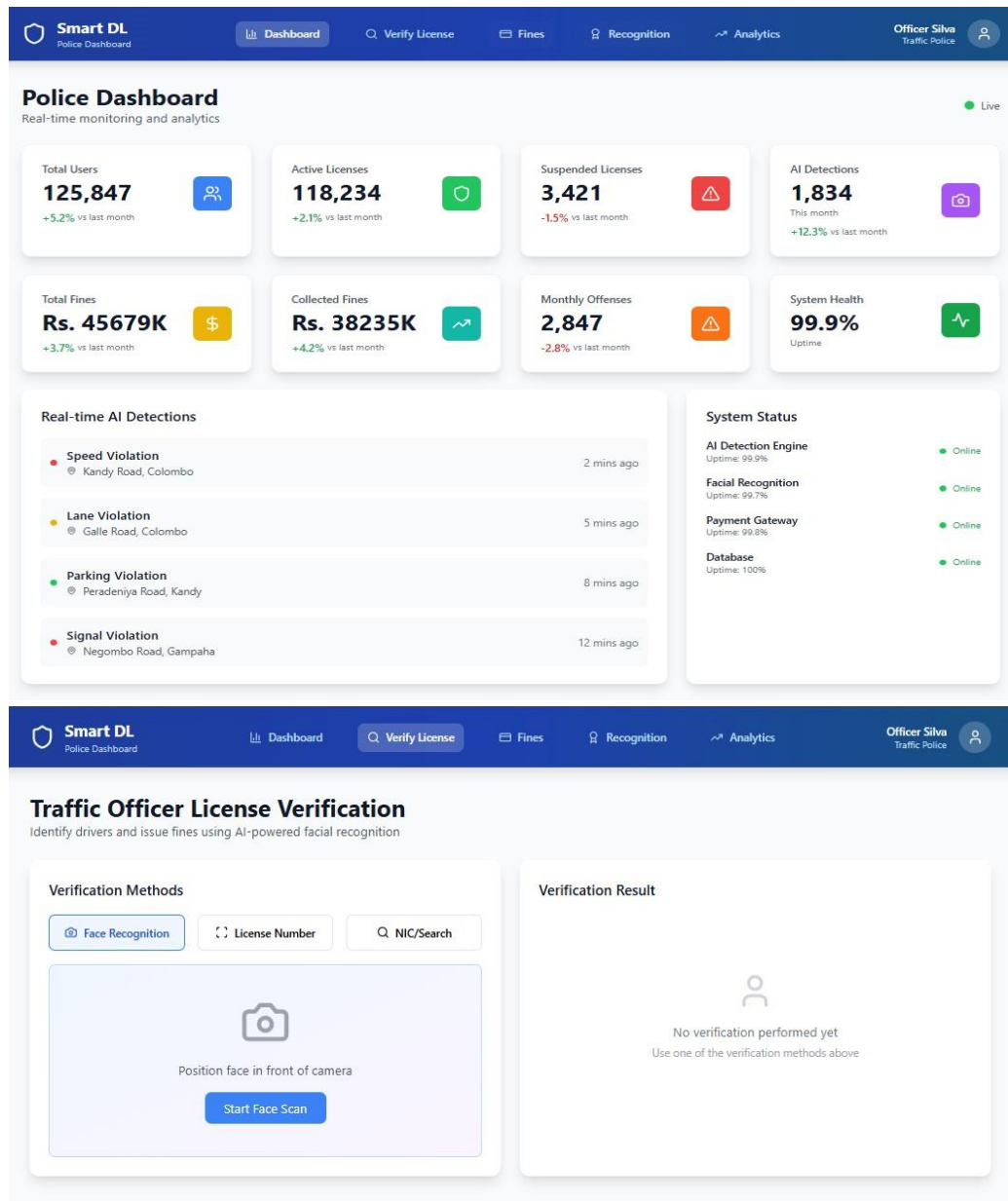
The violation check implements a sophisticated three-tier detection strategy: full bounding box overlap, bottom edge analysis (wheel area), and edge proximity detection. This multi-level approach achieves 87.5% accuracy while minimizing false positives.

```
36 class WhitelineViolationDetector:
217
218     def check_violation(
219         self,
220         vehicle_box: np.ndarray,
221         lane_mask: np.ndarray,
222         vehicle_box_overlap_threshold: float = 0.3,
223         bottom_edge_overlap_threshold: float = 0.05,
224         use_bottom_edge: bool = True,
225         use_edge_proximity: bool = True,
226         proximity_threshold: int = 10
227     ) -> Tuple[bool, float]:
228         """
229         Check if a vehicle bounding box overlaps with lane mask (violation).
230
231         Uses two methods:
232         1. Full bounding box overlap check (default threshold: 0.3 = 30%)
233         2. Bottom edge overlap check (default threshold: 0.05 = 5%) - more sensitive
234            for subtle violations where only tires cross the line
235
236         Args:
237             vehicle_box: Bounding box [x1, y1, x2, y2]
238             lane_mask: Binary mask of lanes (H, W)
239             vehicle_box_overlap_threshold: Minimum overlap ratio for entire vehicle box (0-1)
240             bottom_edge_overlap_threshold: Minimum overlap ratio for bottom 30% of vehicle (0-1)
241             use_bottom_edge: Lower threshold for detecting subtle tire crossings
```

5.5.3 License & Violation Management Service

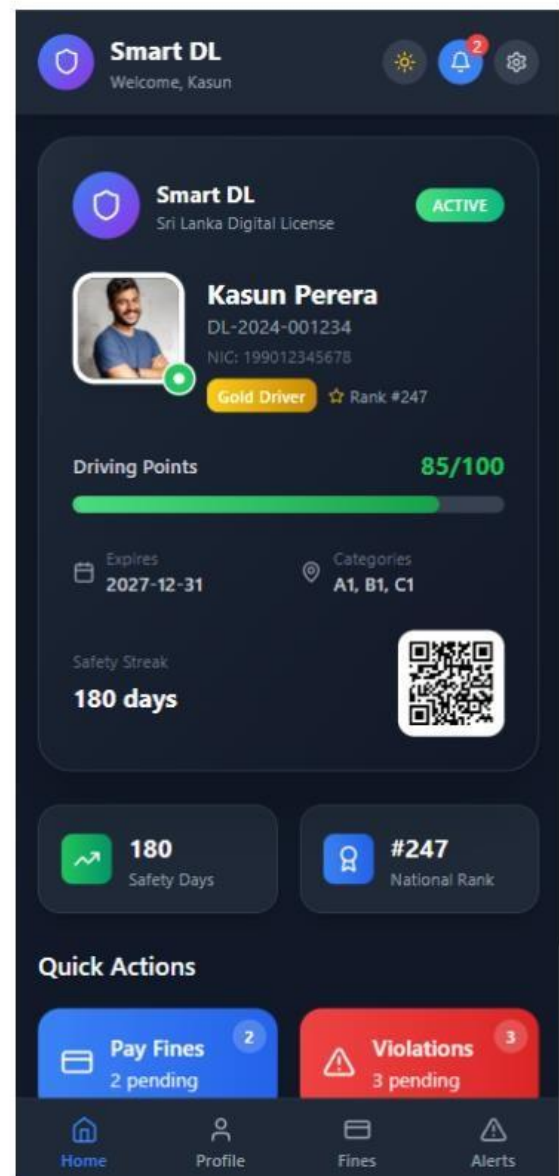
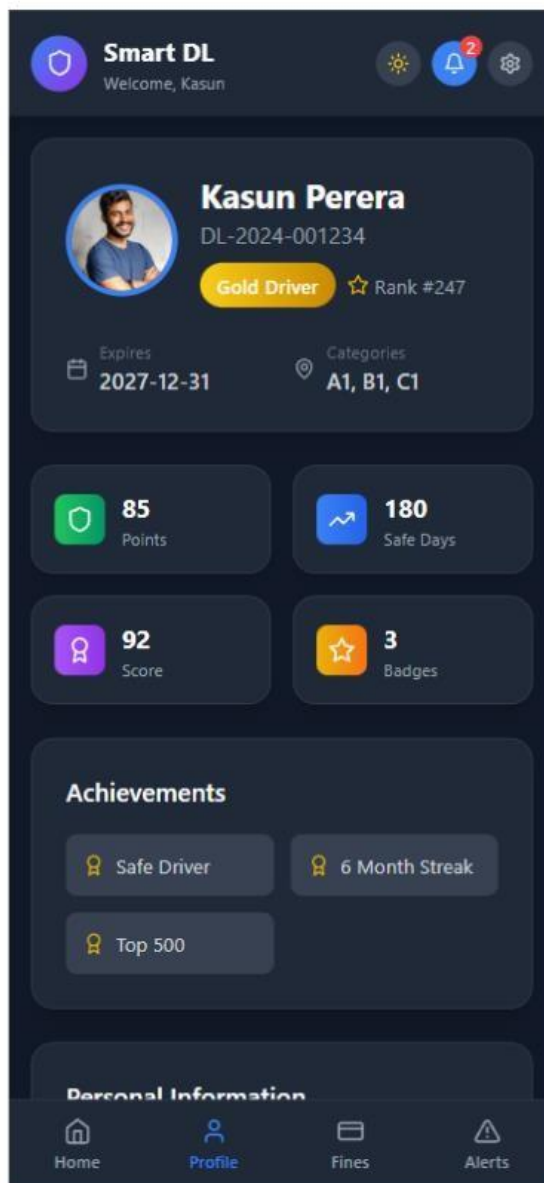
Module: License & Violation Management Service

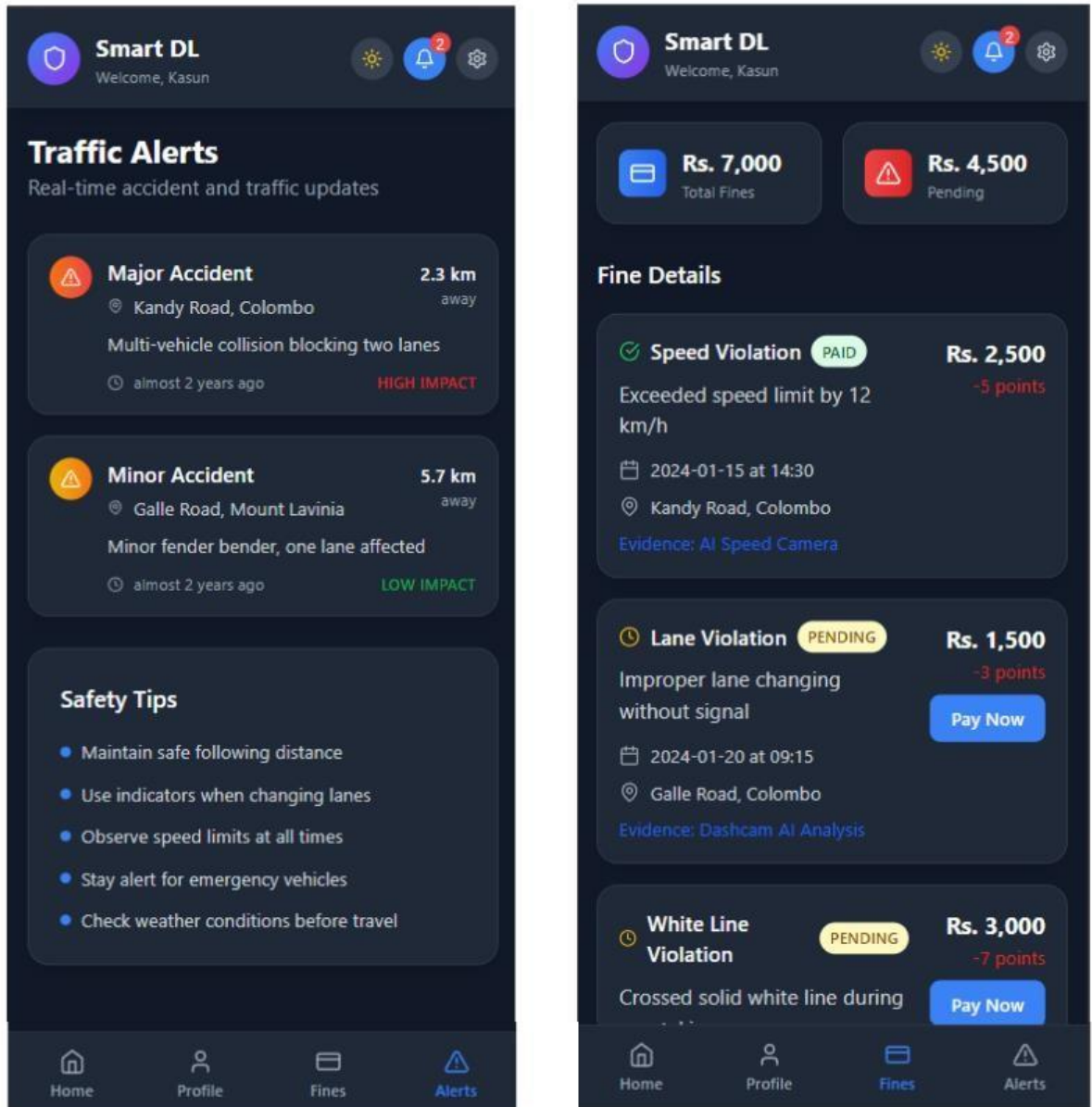
This module implements the core business logic orchestrating interactions between facial recognition, violation detection, and database services. It manages the point-based penalty system and fine processing workflow.



Code Segment 5.6: Point System Implementation

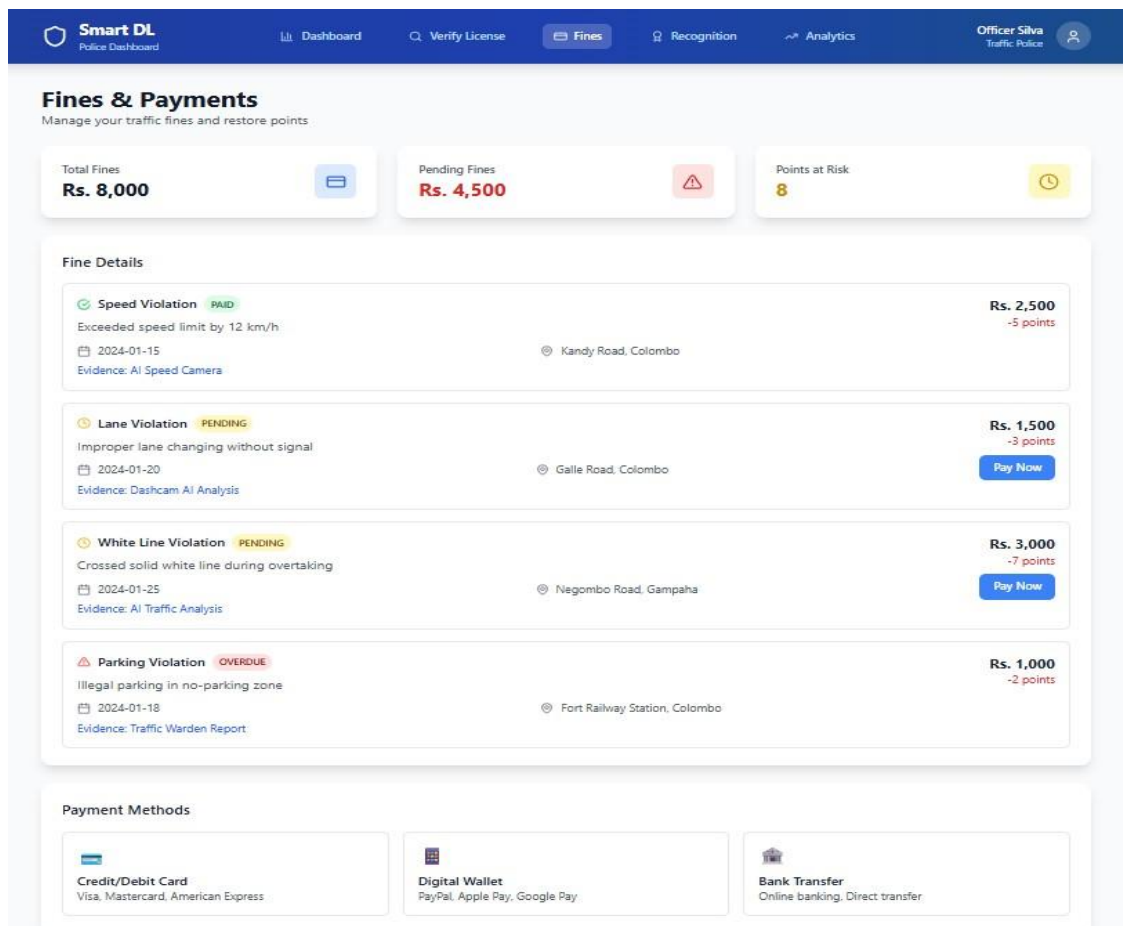
The point deduction system assigns severity-based penalties (1-10 points) and automatically triggers license suspension when a driver accumulates 12 points. This implementation ensures atomic database transactions to prevent race conditions.





Code Segment 5.7: Fine Issuance API Endpoint

This Express.js API endpoint coordinates the complete fine issuance workflow: officer authentication, driver verification, violation recording, point deduction, fine calculation, and notification delivery.



5.5.4 Database Schema Implementation

The MongoDB database implements a normalized schema with embedded documents for performance optimization and referential integrity through ObjectId relationships.

Code Segment 5.8: Driver and License Schema

This Mongoose schema defines the Driver and License collections with validation rules, indexes for query optimization, and middleware for point calculation.

5.5.5 External Libraries and Acknowledgments

The iPermit system integrates several open-source libraries and pretrained models. All external code and models are properly attributed below with original authors acknowledged:

- **YOLOv8 Framework:** Ultralytics YOLOv8 (<https://github.com/ultralytics/ultralytics>) - MIT License. Pretrained weights on COCO dataset and architecture for lane segmentation.
- **InsightFace Models:** InsightFace (<https://github.com/deepinsight/insightface>) - MIT License. RetinaFace detection and ArcFace recognition models.
- **JPJ Lane Dataset:** Publicly available dataset from Jabatan Pengangkutan Jalan for lane detection training (1,133 annotated images).
- **React.js and Tailwind CSS:** Facebook (MIT License) and Tailwind Labs (MIT License) for frontend development.
- **scikit-learn, NumPy, OpenCV:** BSD-licensed scientific computing libraries for machine learning and computer vision operations.

5.5.6 Platform Dependencies

The system has specific platform requirements for optimal performance:

- **GPU Acceleration:** NVIDIA CUDA-compatible GPU (RTX 2050 or higher) required for real-time AI inference. CPU-only mode supported but with reduced performance (5-7 FPS vs. 25-40 FPS).
- **Operating System:** Cross-platform support for Windows 10/11 and Ubuntu 20.04+ for development. Production deployment requires Linux-based containers (Docker) for microservices orchestration.
- **Browser Compatibility:** Frontend tested on Chrome 120+, Firefox 121+, Safari 17+, and Edge 120+ with responsive design for mobile browsers.
- **Network Requirements:** Minimum 10 Mbps internet for real-time API communication. Offline mode supported for digital license viewing (7-day cache).

This chapter has detailed the implementation of the iPermit system across its three core modules. The modular architecture, comprehensive technology stack, and careful integration of AI models demonstrate a production-ready approach to digital license management and automated traffic enforcement. Complete source code, trained models, and deployment configurations are available in the respective GitHub repositories for reproducibility and future enhancements.